

Face detection & recognition (Codsoft AI Internship)

Code explanation

Haar Cascade

I have written a classic OpenCV face detection script. I'll explain it in very simple, line-by-line language so you can confidently explain it in interviews

What this code does (1 line)

It opens your webcam → detects faces → draws boxes → shows live video

Line-by-line explanation

1. Import library

```
import cv2
```

You import OpenCV

This library is used for image & video processing

2. Load face detection model

```
face_cascade = cv2.CascadeClassifier(  
    cv2.data.harcascades + 'haarcascade_frontalface_default.xml'  
)
```

This loads a pre-trained face detection model

- It is called Haar Cascade
- It already knows what a face looks like
- Stored as an .xml file

OpenCV provides these trained models to detect objects like faces

3. Start webcam

```
cap = cv2.VideoCapture(0)
```

0 means default webcam

This starts video capture from your camera

4. Infinite loop (runs continuously)

while True:

Keeps capturing frames again and again
Like a live video stream

5. Read frame from webcam

```
ret, frame = cap.read()
```

frame = current image from camera
ret = True/False (whether camera worked)

6. Convert to grayscale

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

Converts color image → black & white

Why?

- Faster processing
 - Haar Cascade works better on grayscale
-

7. Detect faces

```
faces = face_cascade.detectMultiScale(  
    gray,  
    scaleFactor=1.3,  
    minNeighbors=5  
)
```

This is the main logic

It scans the image and finds faces

Parameters:

scaleFactor=1.3

Looks for faces at different sizes

minNeighbors=5

Removes false detections (improves accuracy)

It returns rectangles (x, y, width, height) for each face

8. Draw rectangle around face

for (x, y, w, h) in faces:

```
    cv2.rectangle(frame, (x,y), (x+w,y+h), (255,0,0), 2)
```

For every detected face:

- Draw a box around it

(255,0,0) = blue color

2 = thickness

9. Show output window

```
cv2.imshow("Face Detection", frame)
```

Displays the video with rectangles

10. Exit condition

```
if cv2.waitKey(1) & 0xFF == ord('q'):  
    break
```

If you press q, program stops

11. Clean up

```
cap.release()  
cv2.destroyAllWindows()
```

Release camera

Close all windows

Final flow (easy to remember)

Camera ON



Capture frame



Convert to grayscale



Detect faces



Draw box



Show result



Repeat

This code uses OpenCV's Haar Cascade classifier to detect faces in real-time from a webcam. Each frame is converted to grayscale, faces are detected using `detectMultiScale`, and bounding boxes are drawn around detected faces.

1. Setting Up the Environment

- **What it does:** This cell installs all the necessary Python libraries that our project needs. Think of it as gathering all the tools you'll use before starting a big project.
- **Key Libraries:** `opencv-python` for image processing, `face_recognition` for recognizing faces, `numpy` for number crunching, `pandas` for handling data, and `ipywidgets` for interactive elements.

2. Importing Essential Modules

What it does: This cell brings specific functions and tools from the installed libraries into our working environment. It's like unpacking your tools and arranging them on your workbench.

- **Examples:** `cv2` (OpenCV) for image/video tasks, `face_recognition` for face handling, `pandas` (as `pd`) for data tables, `matplotlib.pyplot` (as `plt`) for showing images, `datetime` for time and dates, and `google.colab` tools for Colab-specific tasks like file uploads and Google Drive.

3. Setting Up Your Workspace

- **What it does:** This cell connects your Google Colab notebook to your Google Drive, creating specific folders where the program will store known faces, attendance records, and any processed video outputs. It also loads a special file (`haarcascade_frontalface_default.xml`) that helps detect faces.
- **Input/Output:** No direct visual input/output, but it sets up where files will be saved and reads the face detection model.
- **Detection Role:** The `face_cascade = cv2.CascadeClassifier(...)` line is crucial for face detection. It initializes an object that can find faces in an image based on pre-trained patterns.

4. Redundant Google Drive Mount

- **What it does:** This cell attempts to mount Google Drive again. It's often redundant if already mounted in the previous cell and doesn't affect functionality if already connected.

5. Uploading Known Faces

- **What it does:** This function allows you to upload images of people whose faces you want the system to recognize. The filename (without the extension or numbers) will be used as the person's name.
- **Input:** You provide image files through a pop-up file uploader.

- **Output:** The uploaded images are moved into the DATASET_DIR on your Google Drive.

6. Processing Known Faces for Recognition

What it does: This cell processes the images you uploaded. For each image, it finds any faces and creates a unique numerical 'fingerprint' (called an "encoding") for each face. These encodings are stored along with the corresponding names.

- **Input:** Images from the DATASET_DIR.
- **Output:** known_encodings (numerical fingerprints of faces) and known_names (the names associated with those fingerprints). It prints how many unique people were loaded.
- **Recognition Role:** This is where the foundation for face recognition is built. The system learns what each known person's face looks like by creating these unique encodings.

7. Marking Attendance

- **What it does:** This function records when a recognized person is seen. It creates or updates a daily CSV file in the ATTENDANCE_DIR with the person's name and the time they were identified. If a person is already in the file for that day, they won't be added again.
- **Input:** A name (of a recognized person) and the attendance_dir.
- **Output:** A CSV file (e.g., attendance_2024-01-01.csv) containing names and times.

8. Recognizing Faces in a Single Image

- **What it does:** This function takes an image, detects faces within it, compares them to your known_encodings, and identifies any known individuals. It draws a box around each detected face and labels it with the person's name (or "Unknown"). If a known person is found, their attendance is marked. Finally, it displays the image.
- **Input:** An image file (uploaded by you) and the known_encodings and known_names.
- **Detection:** Uses face_cascade.detectMultiScale to find face locations.
- **Recognition:** Uses face_recognition.face_encodings to get encodings of detected faces and face_recognition.compare_faces to match them against known_encodings.
- **Output:** A displayed image with bounding boxes and names, and potentially an updated attendance CSV file.

9. Recognizing Faces in a Video

- **What it does:** Similar to image recognition, but for video. This function reads a video frame by frame, performs face detection and recognition on each frame, and draws boxes/names. It then saves the processed video to an output file. It also attempts to keep track of faces even if detection briefly fails (e.g., if a face quickly turns away).
- **Input:** A video file (uploaded by you) and the known_encodings and known_names.
- **Detection & Recognition:** The same core logic as image recognition, applied to each frame.
- **Output:** A new video file with faces identified and bounding boxes, and potentially an updated attendance CSV file.

10. Real-time Face Recognition with Your Camera

- **What it does:** This advanced cell uses your computer's webcam to capture live video. It continuously processes frames, performing face detection and recognition in real-time. It displays the video feed with identified faces and marks attendance as people are recognized. You can stop it with a 'Quit Camera' button.
- **Input:** Live video stream from your webcam.
- **Detection & Recognition:** Real-time application of the face detection and recognition logic.
- **Output:** A live video feed displayed in the notebook with recognized faces, and an updated attendance CSV file.

11. Viewing Attendance Records

- **What it does:** This function checks the attendance CSV file for today's date. If it exists, it displays the contents (names and times) directly in the notebook and offers to download the CSV file to your computer.
- **Input:** Reads the attendance CSV file from ATTENDANCE_DIR.
- **Output:** A table of today's attendance in the notebook and a downloadable CSV file.

12. Reloading Known Faces

- **What it does:** This cell simply re-runs the load_known_faces function (from cell 6) to re-process the images in your dataset. This is useful if you've added new known faces and want the system to update its knowledge without restarting the whole notebook.
- **Input:** Images from the DATASET_DIR.
- **Output:** Updated known_encodings and known_names arrays.